

GESP四级直通车

GESP四级大纲考点

知识点	考察频率
指针类型的概念及基本应用	高
函数的定义与调用，形参与实参、作用域	高
函数参数传递的概念（值传递，引用传递，指针传递）	高
结构体	高
二维数组与多维数组基本应用	高
算法：递推、排序概念和稳定性、冒泡排序，插入排序、选择排序	中
简单算法复杂度的估算（含多项式，指数复杂度）	中
文件重定向与文件读写操作	低
异常处理	低

一、指针

知识点精讲

指针也是一个变量，只不过它保存的数据是一个内存地址。

```
1  int a = 5; // 变量中的普通青年
2  int * p = &a; //变量中的文艺青年
3
4  printf("%d\n", a); //输出a的值
5  printf("%p\n", p); //输出a的地址
6
7  a = 10; //你可以直接修改变量的值
8
9  *p = 15; //你也可以按照地址修改变量的值，前面的方式是叫名字
10 printf("%d", a); //输出15，因为a的值已经被指针操作修改
```

例题解析

2025年3月 第 3 题 执行下述代码后，变量 a 的值为（ ）。

```
1  int a = 10;
2  int* p = &a;
3  *p = 20
```

- A. 10
- B. 20
- C. 随机值
- D. 编译错误

```
1 double f = 1.5;
2 double * pd = &f
3 printf("%d %d", sizeof p, sizeof pd); // 运行这句话，会发现指针也是占空间的，所占据的空间和
    所指向的类型一样
4
5 char * pc = NULL; //如果先申请要给指针而不去使用，可以让其指向NULL，表示空指针。
```

考察点：

结构体的定义

```
1 struct P{
2     int a;
3     double b;
4     char c;
5 };
6
7 int main() {
8     P z;
9     cin >> z.a >> z.b >> z.c;
10    cout << z.a >> z.b >> z.c;
11    return 0;
12 }
```

例题解析

2025年3月 第 6 题 下面的描述中，（ ）正确定义一个名为 Person 的结构体并正确初始化了一个 Person 结构体的变量 p。

```
1 A.
2 struct Person {
3     string name;
4     int age;
```

```

5  };
6  Person p("Yang", 10);
7
8  B.
9  struct Person {
10     string name,
11     int age;
12 };
13 Person p;
14 p.name = "Yang";
15 p.age = 10;
16
17 C.
18 struct Person {
19     string name;
20     int age;
21 };
22 Person p = { "Yang", 10 };
23
24 D.
25 struct Person {
26     string name;
27     int age;
28 };
29 Person p = new Person("Yang", 10);

```

指针调用成员变量

```

1  #include <iostream>
2  using namespace std;
3
4  // 定义一个结构体
5  struct Person {
6     string name;
7     int age;
8 };
9
10 int main() {

```

```
11    // 创建结构体实例
12    Person person1 = {"Alice", 25};
13
14    // 创建指向结构体的指针
15    Person* ptr = &person1;
16
17    // 使用箭头运算符访问成员
18    cout << "Name: " << ptr->name << endl;
19    cout << "Age: " << ptr->age << endl;
20
21    // 修改成员变量
22    ptr->age = 26;
23    cout << "Updated age: " << ptr->age << endl;
24
25    return 0;
26 }
```

真题复现

2023年6月第3题（14845）下列关于C++语言中指针的叙述，不正确的是()。

- A指针变量中存储的是内存地址。
- B定义指针变量时必须指定其指向的类型。
- C指针变量只能指向基本类型变量，不能指向指针变量。
- D指针变量指向的内存地址不一定能够合法访问。

2023年6月第8题（14850）一个变量定义为`int *p = nullptr`，则下列说法正确的是()。

- A该指针变量的类型为`int`。
- B该指针变量指向的类型为`int`。
- C该指针变量指向的内存地址是随机的。
- D访问该指针变量指向的内存会出现编译错误。

2023年9月第4题（16028）下列关于C++语言中指针的叙述，不正确的是（）。

- A可以定义指向 int 类型的指针。
- B可以定义指向自定义结构体类型的指针。
- C自定义结构体类型可以包含指针类型的元素。
- D不能定义指向void类型的指针，那没有意义。

2023年9月第9题（16033）如果 n 为 int 类型的变量，一个指针变量定义为 `int *p = &n;`，则下列说法正确的是（）。

- A指针变量 p 的值与变量 n 是相同的。
- B指针变量 p 的值与变量 n 的地址是相同的。
- C指针变量 p 指向的值为 'n'。
- D指针变量 p 指向的值与变量 n 的地址是相同的。

2023年9月第16题（16040）在C++语言中，指针变量在逻辑上指向另一个变量在内存中的位置，指针变量本身不占用内存。

- A正确
- B错误

二、二维数组

数组是一种高效的数据存储的方式，所有主流的编程语言中都有数组这个数据结构。

知识点精讲

-多维数组所占用空间

```
1 //1.二维数组的定义
2 int a[10][10]; // 此时数组所占空间为 10 * 10 * 4 一个int4个字节
3 //2.数组空间
4 printf("%d", sizeof a);
5 //3.数组存储连续特性
6 for (int i = 0; i < 10; i++) {
7     for (int j = 0; j < 10; j++) {
8         printf("a[%d,%d]的地址是%p", i, j, &a[i][j]);
9     }
10    puts("");
11 }
12
```

例题解析

2025年3月 第9题 下面（ ）正确定义了二维数组。

- 1 A. `int arr[3,4];`
- 2 B. `int arr[3][4];`
- 3 C. `int arr(3,4);`
- 4 D. `int a[3-4];`

真题复现

2023年6月第4题下列关于 C++语言中数组的叙述，不正确的是()。

- A一维数组在内存中一定是连续存放的。
- B二维数组是一维数组的一维数组。
- C二维数组中的每个一维数组在内存中都是连续存放的。
- D二维数组在内存中可以不是连续存放的。

2023年6月第7题一个二维数组定义为`double array[3][10];`，则这个二维数组占用内存的大小为()。

- A 30

- B 60
- C 120
- D 240

2023年6月 第9题一个二维数组定义为`int array[5][3];`，则`array[1][2]`和`array[2][1]`在内存中的位置相差多少字节？ ()

- A 2字节。
- B 4字节。
- C 8字节。
- D 无法确定

2023年6月 第11题一个数组定义为`int a[5] = {1,2,3,4,5};`，一个指针定义为`int *p = &a[2];`，则执行`a[1] = *p;`后，数据`a`中的值会变为()。

- A {1,3,3,4,5}
- B {2,2,3,4,5}
- C {1,2,2,4,5}
- D {1,2,3,4,5}

2023年6月 第14题执行以下C++语言程序后，输出结果是 ()。

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int array[3][3];
5      for (int i = 0; i < 3; i++)
6          for (int j = 0; j < 3; j++)
7              array[i][j] = i * 10 + j;
8      int sum;
9      for (int i = 0; i < 3; i++)
10         sum += array[i][i];
11     cout << sum << endl;
12     return 0;
13 }
```

- A 3
- B 30
- C 33
- D 无法确定

2023年6月第19题 在C++语言中，可以定义四维数组，但在解决实际问题时不可能用到。因为世界是三维的。

- A 正确
- B 错误

2023年6月第22题 如果希望记录10个最长为99字节的字符串，可以将字符串数组定义为char s[100][10]

- A 正确
- B 错误

2023年9月三级第5题 如果数组定义为 long long array[] = {3, 5, 7, 2}; ，则数组 array 占用的字节数为（ ）。

- A 32
- B 16
- C 8
- D 4

2023年9月三级第7题 以下数组定义，符合C++语言语法的是（ ）。

- A double a[];
- B double b[] = {1, 2.0, '3'};
- C double c[3.0];
- D double[] d = new double[3];

2023年9月第5题（16029）下列关于C++语言中数组的叙述，不正确的是（ ）。

- A 一维数组可以用来表示数列。
- B 二维数组可以用来表示矩阵。
- C 三维数组可以用来表示空间中物体的形状。
- D 世界是三维的，所以定义四维数组没有意义。

2023年9月第8题 一个二维数组定义为 `char array[3][10];`，则这个二维数组占用内存的大小为（ ）。

- A 10
- B 30
- C 32
- D 48

2023年9月第10题 一个三维数组定义为 `long long array[6][6][6];`，则 `array[1][2][3]` 和 `array[3][2][1]` 在内存中的位置相差多少字节？（ ）

- A 70字节
- B 198字节
- C 560字节
- D 无法确定

2023年9月第12题 一个数组定义为 `int a[5] = {1, 2, 3, 4, 5};`，一个指针定义为 `int * p = &a[2];`，则执行 `*p = a[1];` 后，数组a中的值会变为（ ）。

- A {1, 2, 2, 4, 5}
- B {1, 3, 3, 4, 5}
- C {1, 2, 3, 3, 5}
- D {1, 2, 4, 4, 5}

2023年9月第12题 如果希望记录10个最长为99字节的字符串，可以将字符串数组定义为 `char s[10][100];`。

- A 正确
- B 错误

三、函数

函数在数学中的意义是映射，在程序设计中可以看作一个有特定功能的可重复利用的代码块。

知识点精讲

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  //函数的类型为int,因为函数的功能是计算2数之和，结果显然也是整数，我们管结果叫返回值，返回值的类型就是函数的类型
5  //( )是参数表，每个参数都必须有类型，在函数定义部分写的参数叫形式参数
6  //变量a,b定义在函数add内，那么出了add就不能用，这称为作用域，同理，定义在主函数里的res在add里用不了
7  int add(int a, int b) {
8      return a + b;
9  }
10
11 int main() {
12     int res = add(5, 3); //在函数调用里写的参数叫实际参数
13     printf("%d", res);
14     return 0;
15 }
```

例题解析

2025年3月 第 1 题 关于下述代码，说法错误的是（ ）。

```
1  int multiply(int x, int y);
2  int main() {
3      int a = 4;
4      int b = 5;
5      int result = multiply(a, b);
6      std::cout << "The result is: " << result << std::endl;
7      return 0;
8  }
9  int multiply(int x, int y) {
10     return x * y;
11 }
```

A. 函数 multiply 的定义应该放到函数 main 之前。

B. 函数声明 int multiply(int x, int y); 中明确指定了函数 multiply() 的返回值为整数类型。

C. 在 main 函数中，函数 multiply 通过 multiply(a, b) 被调用，其中 a 和 b 是定义在 main 函数中的变量，它们作为实参传递给了 multiply 函数的形参 x 和 y。

D. 运行上述代码，将输出 The result is: 20。

传值

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void swap(int a, int b) { // 这种写法是传递传递参数的值
5      int t = b;
6      b = a;
7      a = t;
8  }
9
10 int main() {
11     int a = 5, b = 3;
```

```
12     swap(a, b);
13     printf("a = %d b = %d", a, b); // 这时候输出5, 3;
14     return 0;
15 }
```

传地址

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void swap(int *a, int *b) { // 这种写法是传递传递参数的值
5      int t = *b;
6      *b = *a;
7      *a = t;
8  }
9
10 int main() {
11     int a = 5, b = 3;
12     swap(&a, &b);
13     printf("a = %d b = %d", a, b); // 这时候输出3, 5;
14     return 0;
15 }
```

传引用

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void swap(int &a, int &b) { // 这种写法是传递传递参数的值
5      int t = b;
6      b = a;
7      a = t;
8  }
9
10 int main() {
11     int a = 5, b = 3;
12     swap(a, b);
```

```
13     printf("a = %d b = %d", a, b); // 这时候输出3, 5;
14     return 0;
15 }
```

例题解析

2025年3月 第 4 题 以下哪种参数传递方式可以避免拷贝大型对象？

- A. 只能用值传递
- B. 只能用引用传递
- C. 只能用指针传递
- D. 引用传递和指针传递均可

```
1  #include <iostream>
2  using namespace std;
3
4  /*传二维数组*/
5
6  //第1种方式：传数组,第二维必须标明
7  /*void display(int arr[][4])*/
8  void display1(int arr[][4],const int irows)
9  {
10     for (int i=0;i<irows;++i)
11     {
12         for(int j=0;j<4;++j)
13         {
14             cout<<arr[i][j]<<" ";    //可以采用parr[i][j]
15         }
16         cout<<endl;
17     }
18     cout<<endl;
```

```

19 }
20
21 //第2种方式：一重指针，传数组指针，第二维必须标明
22 /*void display(int (*parr)[4])*/
23 void display2(int (*parr)[4],const int irows)
24 {
25     for (int i=0;i<irows;++i)
26     {
27         for(int j=0;j<4;++j)
28         {
29             cout<<parr[i][j]<<" ";    //可以采用parr[i][j]
30         }
31         cout<<endl;
32     }
33     cout<<endl;
34 }
35 //注意：parr[i]等价于*(parr+i)，一维数组和二维数组都适用
36
37 //第3种方式：传指针，不管是几维数组都把他看成是指针
38 /*void display3(int *arr)*/
39 void display3(int *arr,const int irows,const int icols)
40 {
41     for(int i=0;i<irows;++i)
42     {
43         for(int j=0;j<icols;++j)
44         {
45             cout<<*(arr+i*icols+j)<<" ";    //注意:(arr+i*icols+j),不是(arr+i*irows+j)
46         }
47         cout<<endl;
48     }
49     cout<<endl;
50 }
51
52
53 //第2种方式：一重指针，传数组指针void display(int (*parr)[4])
54 //缺陷：需要指出第二维大小
55 typedef int parr[4];
56 void display(parr *p)

```

```

57 {
58     int *q=*p;          //q指向arr的首元素
59     cout<<*q<<endl;    //输出0
60 }
61
62 typedef int (*parr1)[4];
63 void display1(parr1 p)
64 {
65     cout<<(*p)[1]<<endl; //输出1
66     cout<<*p[1]<<endl;   //输出4,[]运算符优先级高
67 }
68 //第3种方式:
69 void display2(int **p)
70 {
71     cout<<*p<<endl;      //输出0
72     cout<<*((int*)p+1+1)<<endl; //输出2
73 }
74 */
75
76 int main()
77 {
78     int arr[][4]={0,1,2,3,4,5,6,7,8,9,10,11};
79     int irows=3;
80     int icols=4;
81     display1(arr,irows);
82     display2(arr,irows);
83
84     //注意(int*)强制转换.个人理解: 相当于将a拉成了一维数组处理。
85     display3((int*)arr,irows,icols);
86     return 0;
87 }

```

真题复现

2025年3月 第2题 执行下述代码将输出 ()。

```
1  int x = 10;
2  void func() { int x = 20; std::cout << x; }
3  int main() {
4      func();
5      std::cout << x;
6      return 0;
7  }
```

A. 2020 B. 2010 C. 1010 D. 编译错误

2025年3月 第 5 题 执行下述代码，将输出（ ）。

```
1  void swap(int a, int &b) {
2      int temp = a;
3      a = b;
4      b = temp;
5  }
6  int main() {
7      int x = 1, y = 2;
8      swap(x, y);
9      std::cout << x << y;
10     return 0;
11 }
```

A. 12

B. 21

C. 22

D. 11

2023年6月第5题 下列关于C++语言中函数的叙述，正确的是()。

- A 函数必须有名字。
- B 函数必须有参数。
- C 函数必须有返回值。

- D 函数定义必须写在函数调用前。

2023年6月第12题 以下哪个函数声明在调用时可以传递二维数组的名字作为参数? ()

- A void BubbleSort(int a[][4]);
- B void BubbleSort(int a[3][]);
- C void BubbleSort(int a[][]);
- D void BubbleSort(int ** a);

2023年6月第13题 在下列代码的横线处填写 ()，可以使得输出是“20 10”。

```
1  #include <iostream>
2  using namespace std;
3  void xchg(_____) {          // 在此处填入代码
4      int t = *x;
5      *x = *y;
6      *y = t;
7  }
8  int main(){
9      int a = 10, b = 20;
10     xchg(&a, &b);
11     cout << a << " " << b << endl;
12     return 0;
13 }
14
```

- A int x, int y
- B int *x, int *y
- C int a, int b
- D int &a, int &b

2023年6月第18题 在C++语言中，函数的参数默认以引用传递方式进行传递。

- A 正确
- B 错误

2023年6月第20题 在C++语言中，一个函数没有被调用时，它的参数不占用内存。

- A 正确
- B 错误

2023年6月第21题 在C++语言中，如果一个函数可能抛出异常，那么一定要在try子句里调用这个函数。

- A 正确
- B 错误

2023年9月第6题 下列关于C++语言中函数的叙述，正确的是（）。

- A 函数调用前必须定义。
- B 函数调用时必须提供足够的实际参数。
- C 函数定义前必须声明。
- D 函数声明只能写在函数调用前。

2023年9月第19题 在C++语言中，在函数调用时，通过引用传递的参数不会复制实际参数，因此不会额外占用内存。

- A 正确
- B 错误

四、排序算法

GESp四级要求掌握的排序算法共3种：选择、插入、冒泡。

考察点：

-算法模板

-排序稳定性：选择不稳定，其他稳

-时间复杂度：都是 $O(n^2)$

```
1 //冒泡排序 稳定
2 void bubbleSort(int *a, int n) {
```

```

3     for (int i = 0; i < n - 1; i++) {
4         bool flag = false;
5         for (int j = 0; j < n - 1 - i; j++) {
6             if (a[j] > a[j + 1]) {
7                 std::swap(a[j], a[j + 1]);
8                 flag = true;
9             }
10        }
11        if (flag == false) break;
12    }
13 }
14 //插入排序 稳定
15 void insertionSort(int *a, int n) {
16     for (int i = 1; i < n; i++) {
17         int tmp = a[i], j = i;
18         for ( ; j > 0 && tmp < a[j - 1]; j--) {
19             a[j] = a[j - 1];
20         }
21         a[j] = tmp;
22     }
23 }
24 //选择排序 不稳定
25 void selectSort(int *a, int n) {
26     for (int i = 0; i < n; i++) {
27         int minId = i;
28         for (int j = i + 1; j < n; j++) {
29             if (a[j] < a[minId]) minId = j;
30         }
31         std::swap(a[i], a[minId]);
32     }
33 }

```

2023年6月第15题（14857）在下列代码的横线处填写()，完成对有n个int类型元素的数组array由小到大排序。

```

1 void SelectionSort(int array[], int n){
2     int i,j,min,temp;
3     for (i = 0; i < n - 1;i++){
4         min = i;
5         for (j = i + 1; j < n; j++)
6             if (_____) // 在此处填入代码
7                 min = j;
8         temp = array[min];
9         array[min] = array[i];
10        array[i] = temp;
11    }
12 }
13

```

- A array[min] > array[j]
- B array[min] > array[i]
- C min > array[j]
- D min > array[i]

2023年9月第15题（16039）在下列代码的横线处填写（），完成对有 n 个 int 类型元素的数组 array 由小到大排序。

```

1 void BubbleSort(int array[],int n){
2     for (int i = n; i >= 2; i--) // n-1循环 i=n, i>=2 够n-1次
3         for (_____) // 在此处填入代码
4             if (array[j] > array[j + 1]){
5                 int t = array[j];
6                 array[j] = array[j + 1];
7                 array[j + 1] = t;
8             }
9 }

```

- A int j = 1; j < n; j++
- B int j = 0; j < n; j++
- C int j = 0; j < i - 1; j++

- D `int j = 0; j < i; j++`

2023年9月第17题（16041）对 N 个元素的数组执行插入排序算法，通常的时间复杂度是 $O(N^2)$ 。

- A 正确
- B 错误

五、次重点

5.1 重定向输入输出

C++程序默认标准输入输出，即小黑窗口，可以使用`freopen`语句改为文件读写。

```
1
2  int main() {
3      freopen("xxx.in", "r", stdin);
4      freopen("xxx.out", "w", stdout);
5      //balabala
6      return 0;
7  }
```

例题解析

第 15 题 下面哪种方式不能实现将字符串 "Happy Spring!" 输出重定向到文件 log.txt （）。

```
1  A
2  freopen("log.txt", "w", stdout);
3  cout << "Happy Spring!" << endl;
4  fclose(stdout);
5
6  B
7  std::ofstream outFile("log.txt");
8  outFile << "Happy Spring!" << endl;
9  outFile.close();
10
11 C
12 std::ofstream outFile("log.txt");
```

```
13 cout << "Happy Spring!" << endl;
14 outFile.close();
15
16 D
17 ofstream log_file("log.txt");
18 streambuf* org_cout = cout.rdbuf();
19 cout.rdbuf(log_file.rdbuf());
20 cout << "Happy Spring!" << endl;
21 cout.rdbuf(org_cout);
```

5.2 异常处理

异常是程序在执行期间产生的问题。C++ 异常是指在程序运行时发生的特殊情况，比如尝试除以零的操作。

异常提供了一种转移程序控制权的方式。C++ 异常处理涉及到三个关键字：**try**、**catch**、**throw**。

- **throw**: 当问题出现时，程序会抛出一个异常。这是通过使用 **throw** 关键字来完成的。
- **catch**: 在您想要处理问题的地方，通过异常处理程序捕获异常。**catch** 关键字用于捕获异常。
- **try**: **try** 块中的代码标识将被激活的特定异常。它后面通常跟着一个或多个 **catch** 块。

如果有一个块抛出一个异常，捕获异常的方法会使用 **try** 和 **catch** 关键字。**try** 块中放置可能抛出异常的代码，**try** 块中的代码被称为保护代码。使用 **try/catch** 语句的语法如下所示：

```
1 double division(int a, int b)
2 {
3     if( b == 0 )
4     {
5         throw "Division by zero condition!";
6     }
7     return (a/b);
8 }
```

catch 块跟在 **try** 块后面，用于捕获异常。您可以指定想要捕捉的异常类型，这是由 **catch** 关键字后的括号内的异常声明决定的。

```
1  try
2  {
3      // 保护代码
4  }catch( ExceptionName e )
5  {
6      // 处理 ExceptionName 异常的代码
7  }
```

一个实例：

```
1  #include <iostream>
2  using namespace std;
3
4  double division(int a, int b) {
5      if( b == 0 ) {
6          throw "Division by zero condition!";
7      }
8
9      return (a/b);
10 }
11
12 int main () {
13     int x = 50;
14     int y = 0;
15     double z = 0;
16
17     try {
18         z = division(x, y);
19         cout << z << endl;
20     } catch (const char * msg) {
21         cout << msg << endl;
22     }
23     return 0;
24 }
```

5.3 递推

考察点：

-斐波那契数列

```
1 void init() {
2     fibo[1] = 1, fibo[2] = 1;
3     for (int i = 3; i <= 30; i++) fibo[i] = fibo[i-1] + fibo[i-2];
4 }
```

六、编程题

二维数组模拟

2025年3月

荒地开垦

小杨有一大片荒地，可以表示为一个 行 列的网格图。小杨想要开垦这块荒地，但荒地中一些位置存在杂物，对于一块不存在杂物的荒地，该荒地可以开垦当且仅当其上 下左右四个方向相邻的格子均不存在杂物。小杨可以选择至多一个位置，清除该位置的杂物，移除杂物后该位置变为荒地。小杨想知道在清除至多一个位置的 杂物的情况下，最多能够开垦多少块荒地。

二阶矩阵

小 A 有一个 行 列的矩阵。小 A 认为一个 的矩阵 是好的，当且仅当 。其中 表示矩阵 的第 行第 列 的元素。小 A 想知道 中有多少个好的子矩阵。

6.1 进制转换

```
1 //洛谷P1143
2 #include <bits/stdc++.h>
3 using namespace std;
```

```
4
5 char a[100];
6 char t[] = "0123456789ABCDEF";
7
8 int main () {
9     int k, s, res = 0, j = 1;
10    scanf("%d %s %d", &k, a, &s);
11    int L = strlen(a);
12    for (int i = L - 1; i >= 0; i--) {
13        if (a[i] >= '0' && a[i] <= '9') {
14            res += (a[i] - '0') * j;
15        } else {
16            res += (a[i] - 'A' + 10) * j;
17        }
18        j *= k;
19    }
20    char stk[105];
21    int top = 0;
22    while (res) {
23        stk[++top] = t[res%s];
24        res /= s;
25    }
26    while (top > 0) {
27        printf("%c", stk[top--]);
28    }
29    return 0;
30 }
```

6.2 数字拆分和重组

```
1 //将x的各个数位拆出
2 while (x) {
3     stk[++top] = x % 10;
4     x /= 10;
5 }
6
7 //将x重组为倒序 -1230 -> -321
8 int res = 0;
9 while (x) {
10     res = res * 10 + x % 10;
11     x /= 10;
12 }
13
```